

A1. Enterprise Applications & Spring Boot [5 marks]

Question A1 [5 marks]

Describe three characteristics that distinguish an **enterprise application** from a standard desktop application. For each characteristic, give a concrete example of how Spring Boot addresses it. [5 marks]

Student answer space:

Three Characteristics and how Spring Boot addresses them:

1. **Scalability:** Enterprise applications must handle high traffic and thousands of concurrent users.
 - **Spring Boot Solution:** Spring Boot is designed for cloud-native development. Through Spring Cloud, it provides tools for service discovery, load balancing, and circuit breakers, making it easy to scale horizontally.
2. **Security:** These apps deal with sensitive data and require complex authentication/authorization (RBAC).
 - **Spring Boot Solution:** Spring Security provides a robust framework that integrates easily with standards like OAuth2, JWT, and LDAP, ensuring that different parts of the application are restricted to authorized users only.
3. **Third-Party Integration:** Enterprise apps must connect to various databases, legacy systems, and messaging brokers.
 - **Spring Boot Solution:** The use of "Starter" dependencies (e.g., spring-boot-starter-data-jpa, spring-boot-starter-amqp) provides pre-configured templates that allow developers to connect to databases (SQL/NoSQL) and messaging systems (Kafka/RabbitMQ) with minimal code.

A2. Monolith vs Microservices [5 marks]

Question A2 [5 marks]

A start-up is building a food delivery platform. The team has 3 developers and a 3-month deadline. **(a)** Recommend an appropriate architecture (monolith or microservices) and justify your choice with two specific reasons. **(b)** Describe the **Strangler Fig pattern** and explain when it would be appropriate to migrate from the initial architecture to microservices. [5 marks]

Student answer space:

(a) Recommendation: Monolithic Architecture

- **Reason 1 (Development Speed):** With only 3 developers and a 3-month deadline, a monolith is much faster to build, test, and deploy. There is no need to worry about the overhead of inter-service communication or distributed debugging.
- **Reason 2 (Operational Simplicity):** A small team cannot afford the "infrastructure tax" of microservices (e.g., managing service registries, API gateways, and distributed logging). A monolith is a single unit that is easier to manage.

(b) Strangler Fig Pattern

- **Description:** This is a migration strategy where you gradually replace parts of a legacy system (the monolith) with new microservices. A facade or API Gateway is used to route traffic to the new services for specific features, while the rest remains in the monolith. Eventually, the new services "strangle" and replace the old system entirely.
- **When to Migrate:** It is appropriate to migrate when the monolith becomes a "Big Ball of Mud" (too complex to change), when development speed slows down due to the size of the codebase, or when specific modules (like "Payments") need to scale independently from the rest of the app.

A3. SOLID Principles [5 marks]

Question A3 [5 marks]

The following class violates TWO of the SOLID principles. Identify both violations by name, explain why each is a violation, and briefly describe how you would refactor the class to fix each one. [5 marks]

```

public class OrderProcessor {

    private final List<Order> orders = new ArrayList<>();

    public void addOrder(Order order) {
        orders.add(order);
    }

    public void processOrders() {
        for (Order order : orders) {
            // Step 1: validate
            if (order.getTotal() <= 0) throw new RuntimeException("Invalid order");

            // Step 2: charge credit card
            String url = "https://payment.example.com/charge";
            HttpClient client = HttpClient.newHttpClient();
            // ... build and send HTTP request ...

            // Step 3: send confirmation email
            Properties props = new Properties();
            props.put("mail.smtp.host", "smtp.gmail.com");
            // ... build and send SMTP email ...

            // Step 4: write to PostgreSQL
            Connection conn = DriverManager.getConnection("jdbc:postgresql://..."); // ... execute SQL INSERT ...
        }
    }
}

```

Student answer space:

The OrderProcessor class (based on standard examples where this class also handles saving and emailing) violates:

1. Violation: Single Responsibility Principle (SRP)

- **Why:** If the OrderProcessor handles adding orders, calculating totals, saving to a database, and sending emails, it has too many reasons to change. A class should only have one responsibility.
- **Refactor:** Create separate classes: an OrderRepository to handle database storage and an EmailService to handle notifications.

2. Violation: Dependency Inversion Principle (DIP)

- **Why:** The class likely depends on concrete implementations (e.g., it might call new MySQLDatabase()) rather than abstractions. This makes the code tightly coupled and hard to test.
- **Refactor:** Use Dependency Injection. Pass an interface (e.g., IOrderRepository) into the constructor so the processor doesn't care which specific database is being used.

A4. Maven Build Lifecycle [5 marks]

Question A4 [5 marks]

(a) List the six core Maven lifecycle phases in the correct order, starting from **validate**. (b) A developer runs `mvn package` and it fails with a compilation error. Which lifecycle phases ran before the failure? (c) Explain the difference between **mvn package** and **mvn install** and when you would use each. [5 marks]

Student answer space:

(a) Six core Maven lifecycle phases (in order):

1. validate (Checks if the project is correct and all information is available)
2. compile (Compiles the source code of the project)
3. test (Runs tests using a suitable unit testing framework)
4. package (Takes the compiled code and packages it in its distributable format, like a JAR)
5. verify (Runs checks on the results of integration tests)
6. install (Installs the package into the local repository for use as a dependency in other projects)

(b) Phases run before a compilation failure:

When running `mvn package`, Maven executes phases sequentially. If it fails at the compile phase, the following phases ran:

- validate
- compile (This is the phase where the failure occurred).

(c) Difference between `mvn package` and `mvn install`:

- `mvn package`: This command compiles your code and bundles it into a shareable format (like a .jar or .war file). The file is placed in the project's /target folder. Use this when you just want to create the final executable file.
- `mvn install`: This does everything the package does, but it also copies the resulting JAR/WAR file into your Local Maven Repository (usually the .m2 folder). Use this when you have multiple projects on your machine and one project needs to use the other as a dependency.

1. validate (Checks if the project is correct and all information is available)

2. compile (Compiles the source code of the project)

3. test (Runs tests using a suitable unit testing framework)

4. package (Takes the compiled code and packages it in its distributable format, like a JAR)

5. verify (Runs checks on the results of integration tests)

6. install (Installs the package into the local repository for use as a dependency in other projects)

(b) Phases run before a compilation failure:

When running the `mvn package`, Maven executes phases sequentially. If it fails at the compile phase, the following phases ran:

- validate
- compile (This is the phase where the failure occurred).

(c) Difference between `mvn package` and `mvn install`:

- `mvn package`: This command compiles your code and bundles it into a shareable format (like a .jar or .war file). The file is placed in the project's /target folder. Use this when you just want to create the final executable file.
- `mvn install`: This does everything the package does, but it also copies the resulting JAR/WAR file into your Local Maven Repository (usually the .m2 folder). Use this when you have multiple projects on your machine and one project needs to use the other as a dependency.

A5. Spring IoC & Dependency Injection [5 marks]

Question A5 [5 marks]

(a) Explain what **Inversion of Control (IoC)** means and how Spring's container implements it. (b) Compare constructor injection vs field injection in Spring. Which is preferred and why? (c) What problem does dependency injection solve when it comes to unit testing? Explain with a brief example. [5 marks]

Student answer space:

A5. Spring IoC & Dependency Injection [5 marks]

(a) Inversion of Control (IoC) & Spring's Implementation

- **Definition:** IoC is a design principle where the control of object creation and lifecycle is transferred from the programmer to a container or framework. Instead of the application code calling a library, the framework calls the application code.
- **Spring's Implementation:** Spring uses the `ApplicationContext` (the IoC Container). It reads configuration metadata (via annotations like `@Component` or XML), instantiates the objects (Beans), wires their dependencies together, and manages them throughout their lifecycle.

(b) Constructor Injection vs. Field Injection

- **Comparison:** Field injection uses `@Autowired` directly on private variables. Constructor injection passes dependencies through a class constructor.
- **Preferred Method:** Constructor Injection is preferred.
- **Why:**
 1. **Immutability:** It allows dependencies to be marked as `final`.
 2. **Testability:** You can easily pass mock objects into the constructor without needing a Spring container.
 3. **Safety:** It prevents `NullPointerException` because the object cannot be created without its required dependencies.

(c) Unit Testing & Dependency Injection

- **Problem Solved:** DI solves Tight Coupling. Without DI, a class creates its own dependencies (using `new`), making it impossible to replace them with "Mocks" during a test.
- **Example:** If a `UserService` creates a real `DatabaseConnection` inside its constructor, you can't test the service without a real database. With DI, you "inject" a mock database, allowing you to test only the logic of the `UserService` in isolation.

A6. Layers of a Spring Boot Application [5 marks]

Question A6 [5 marks]

Draw and label the four standard layers of a Spring Boot application (**Controller** → **Service** → **Repository** → **Database**). For each layer: (a) state its single responsibility, (b) name the Spring annotation(s) used to declare a class in that layer, (c) explain what should **not** live in that layer. [5 marks]

Diagram space (draw or attach):

Diagram Flow:

Client Request → Controller → Service → Repository → Database

Layer	(a) Single Responsibility	(b) Spring Annotation	(c) What should NOT live here
Controller	Handles HTTP requests, validates input, and returns responses (REST API).	@RestController	Business logic or direct database queries.
Service	Contains the core business logic and coordinates multiple repository calls.	@Service	HTTP request handling or raw SQL/database logic.
Repository	Handles data persistence and communication with the database.	@Repository	Business rules or decision-making logic.
Database	Storing and retrieving persistent data.	N/A (External)	Any application-level code or logic.

Section B — Java Fundamentals [40

marks]

Answer all questions. Show your working. Code answers will be tested for correctness.

B1. Java Collections & Generics [8 marks]

Question B1.1 [4 marks]

Study the following code carefully. For each numbered line marked with // ?, write what the output will be. If a line throws an exception, write the exception name and explain why. [4 marks]

```

public static void main(String[] args) {
    List<String> list = new ArrayList<>(List.of("banana", "apple", "cherry"));

    Collections.sort(list);
    System.out.println(list.get(0)); // Line 1 → ?

    Map<String, Integer> scores = new LinkedHashMap<>();
    scores.put("Alice", 90);
    scores.put("Bob", 75);
    scores.put("Alice", 95);
    System.out.println(scores.size()); // Line 2 → ? System.out.println(scores.get("Alice")); // Line 3
    → ?

    Set<Integer> set = new HashSet<>(List.of(1,2,3,2,1));
    System.out.println(set.size()); // Line 4 → ?

    List<Object> items = new ArrayList<>();
    items.add("hello");
    items.add(42);
    String s = (String) items.get(1); // Line 5 → ? }

```

Answers:

Line	Expected Output / Exception	Explanation
1	apple	Collections.sort() sorts strings alphabetically. After sorting "banana", "apple", and "cherry", "apple" becomes the first element (index 0).
2	2	Maps do not allow duplicate keys. The second put("Alice", 95) overwrites the first, resulting in only two entries: Alice and Bob.
3	95	The value for the key "Alice" was updated from 90 to 95 in the third put operation.
4	3	A HashSet only stores unique elements. The input list [1, 2, 3, 2, 1] contains duplicates, so the set only keeps {1, 2, 3}.
5	ClassCastException	The element at items.get(1) is an Integer (42). You cannot cast an Integer object to a String object; they are in different class hierarchies.

Question B1.2 [4 marks]

Complete the following generic method `findMax`. It must work for any type that implements `Comparable`. Then write two test cases demonstrating it with (a) a list of Integers, (b) a list of Strings. [4 marks]

```

public static <T extends Comparable<T>> T findMax(List<T> list) { // TODO: implement — return
the largest element
// throw IllegalArgumentException if list is null or empty

}

// Write your test cases below:

```

Your solution:

```
// (a) Test Case with Integers
List<Integer> intList = List.of(10, 45, 23, 89, 12);
System.o

// (b) Test Case with Strings
List<String> stringList = List.of("Zebra", "Apple", "Mango", "Xylophone");
System.out.println("Max String: " + findMax(stringList)); // Expected: Zebra

public static <T extends Comparable<T>> T findMax(List<T> list) {
    // Check if list is null or empty
    if (list == null || list.isEmpty()) {
        throw new IllegalArgumentException("List cannot be null or empty");
    }

    T max = list.get(0); // Assume first element is max
    for (T element : list) {
        if (element.compareTo(max) > 0) {
            max = element;
        }
    }
    return max; ut.println("Max Integer: " + findMax(intList)); // Expected: 89
}
}
```

1. **Generics:** The `<T extends Comparable<T>>` syntax ensures that the method can only accept types that can be compared to each other (like String, Integer, Double).
2. **Comparison:** Because we use generics, we cannot use the `>` operator. We must use the `.compareTo()` method. If the result is `> 0`, it means the current element is larger than our current max.
3. **Exception Handling:** The method immediately checks for an empty or null list to prevent a `NoSuchElementException` or `NullPointerException`

B2. Object-Oriented Programming [8 marks]

Question B2.1 [5 marks]

Design a small class hierarchy for a ShopWave product catalogue. Requirements: **(a)** Create an abstract class `Product` with fields: `id`, `name`, `price`, `stock`. **(b)** Add a method `calculateDiscount(double percentage)` — abstract. **(c)** Create two concrete subclasses: `PhysicalProduct` (extra field: `weightKg`) and `DigitalProduct` (extra field: `downloadUrl`). **(d)** Implement `calculateDiscount` differently in each subclass. **(e)** Override `toString()` in both subclasses to include all relevant fields. [5 marks]

Your code:

```
abstract class Product {
    protected int id;
    protected String name;
    protected double price;
    protected int stock;

    public Product(int id, String name, double price, int stock) {
        this.id = id;
        this.name = name;
        this.price = price;
        this.stock = stock;
    }
}
```



```

    public abstract double calculateDiscount(double percentage);
}

class PhysicalProduct extends Product {
    private double weightKg;

    public PhysicalProduct(int id, String name, double price, int stock, double weightKg) {
        super(id, name, price, stock);
        this.weightKg = weightKg;
    }

    @Override
    public double calculateDiscount(double percentage) {
        // Physical products get a standard discount
        return this.price * (1 - (percentage / 100));
    }

    @Override
    public String toString() {
        return String.format("PhysicalProduct [ID=%d, Name=%s, Price=%.2f, Stock=%d, Weight=%.2fkg]",
            id, name, price, stock, weightKg);
    }
}

class DigitalProduct extends Product {
    private String downloadUrl;

    public DigitalProduct(int id, String name, double price, int stock, String downloadUrl) {
        super(id, name, price, stock);
        this.downloadUrl = downloadUrl;
    }

    @Override
    public double calculateDiscount(double percentage) {
        // Digital products get an extra 5% off the discount because there is no shipping cost
        return this.price * (1 - ((percentage + 5) / 100));
    }

    @Override
    public String toString() {
        return String.format("DigitalProduct [ID=%d, Name=%s, Price=%.2f, Stock=%d, URL=%s]",
            id, name, price, stock, downloadUrl);
    }
}

```

Question B2.2 [3 marks]

Explain the difference between **method overloading** and **method overriding** in Java. For each: (a) write a two-line code example, (b) state whether it is resolved at compile time or runtime, and (c) give one real-world scenario from a Spring Boot application where you would use it. [3 marks]

Student answer space:

Feature	Method Overloading	Method Overriding
(a) Code Example	void search(String name) { ... } void search(int id) { ... }	@Override void process() { ... }
(b) Resolution	Compile-time (Static binding)	Runtime (Dynamic binding)
(c) Spring Scenario	A Repository where you have multiple findBy... methods with different parameters.	A Custom Service that overrides a method from a base GenericService to provide specific logic.

B3. Java Streams & Functional Programming [8 marks]

Question B3.1 [4 marks]

Rewrite each of the following for-loops as a single Stream pipeline. The behaviour must be identical. [4 marks — 1 per loop]

```
// Loop 1 – collect all names that start with 'A'
List<String> result1 = new ArrayList<>();
for (String name : names) {
    if (name.startsWith("A")) result1.add(name);
}

// Loop 2 – sum all order totals over 100
double total = 0;
```

```
for (Order o : orders) {
    if (o.getTotal() > 100) total += o.getTotal();
}

// Loop 3 – build a map of product name → price
Map<String, Double> priceMap = new HashMap<>();
for (Product p : products) {
    priceMap.put(p.getName(), p.getPrice());
}

// Loop 4 – find the most expensive product (or throw if empty)
Product maxPriceProduct = null;
for (Product p : products) {
    if (maxPriceProduct == null || p.getPrice() > maxPriceProduct.getPrice()) maxPriceProduct = p;
}
if (maxPriceProduct == null) throw new NoSuchElementException();
```

Your Stream solutions:

Loop 1

```
List<String> result1 = names.stream()
    .filter(name -> name.startsWith("A"))
    .collect(Collectors.toList());
```

Loop 2

```
double total = orders.stream()
    .mapToDouble(Order::getTotal)
    .filter(t -> t > 100)
    .sum();
```

Question B3.2 [4 marks]

Study the following Stream pipeline and answer the questions. [4 marks]

```
List<String> result = orders.stream()
    .filter(o -> o.getStatus() == Status.SHIPPED)
    .sorted(Comparator.comparing(Order::getCreatedAt).reversed())
    .limit(5)
    .map(o -> o.getCustomer().getName() + " - " + o.getTotal())
    .collect(Collectors.toList());
```

1. What does this pipeline produce? Describe the result in plain English.
2. Which operations are intermediate and which are terminal? List each operation and classify it.
3. Why is this pipeline lazy? What does that mean for performance?

4. If the orders list has 1 000 000 elements and only 3 have status SHIPPED, how many Order objects does the sorted() operation receive? Explain.

Student answer space:

1. What does this pipeline produce?

It produces a List of Strings containing the names of customers and their order totals (formatted as "Name - Total") for the five most recent orders that have the status SHIPPED.

2. Operations Classification:

- Intermediate Operations: filter(), sorted(), limit(), map(). (These transform the stream but do not trigger execution).
- Terminal Operation: collect(). (This triggers the processing and produces the final result).

3. Why is it lazy? Performance impact?

- Why: Stream operations are lazy because intermediate operations are not executed until the terminal operation (collect) is called.
- Performance: This allows for "Short-circuiting." For example, if the pipeline didn't have a sorted step, Maven could stop processing as soon as it found the first 5 elements. Even with sorting, laziness allows the engine to optimize the data flow into a single pass.

4. Filter Efficiency:

- Answer: Only 3 Order objects will reach the sorted() operation.
- Explanation: The filter() operation is executed first. Since only 3 orders meet the condition o.getStatus() == Status.SHIPPED, all other 999,997 orders are discarded before the sorting logic even begins.

B4. Exception Handling [6 marks]

Question B4.1 [3 marks]

Explain the difference between **checked exceptions**, **unchecked exceptions**, and **errors** in Java. For each: (a) give the root class in the hierarchy, (b) give one concrete example, (c) state whether you must handle it or not and why. [3 marks]

Student answer space:

Type	(a) Root Class	(b) Concrete Example	(c) Must handle? & Why
Checked	Exception	IOException OR SQLException	Yes. The compiler forces you to handle them because they represent recoverable conditions outside the app's control.
Unchecked	RuntimeException	NullPointerException	No. The compiler does not force handling. They usually represent programming bugs that should be fixed rather than caught.
Error	Error	OutOfMemoryError	No. You should not try to catch these. They represent serious JVM-level problems that the application cannot recover from.

Question B4.2 [3 marks]

The code below has **three bugs** related to exception handling. Identify each bug, explain why it is incorrect, and write the corrected version of each flawed section. [3 marks]

```
public String readUserFile(String filename) {  
    FileReader reader = null;  
    try {  
        reader = new FileReader(filename);  
    }  
}
```

```
BufferedReader br = new BufferedReader(reader);  
return br.readLine();  
} catch (Exception e) {  
    // Bug 1: broad catch  
    return null;  
} catch (FileNotFoundException e) {  
    // Bug 2: unreachable catch  
    throw new RuntimeException("File not found: " + filename); } finally {  
    reader.close(); // Bug 3: close without null check & uncaught IOException }  
}
```

Your analysis and fixes:

Bug 1: Resource Leak (Manual Closing)

- **Incorrect:** The `FileReader` is opened but never closed in a `finally` block. This keeps the file handle open in the OS.
- **Correction:** Use `Try-with-resources` (introduced in Java 7) to ensure the reader closes automatically.

```
try (FileReader reader = new FileReader(filename)) { ... }
```

Bug 2: Catching Generic 'Exception'

- **Incorrect:** (Usually found in the catch block) Catching `Exception e` is too broad. It might catch runtime bugs (like `NullPointerException`) that you didn't intend to handle there.
- **Correction:** Catch specific exceptions like `FileNotFoundException` or `IOException`.

Bug 3: Swallowing the Exception / Silent Failure

- **Incorrect:** Many developers leave the catch block empty or just use `e.printStackTrace()` and return `null`. This makes it impossible for the calling method to know the file read failed.

B5. Java Concurrency & Modern Java 21 [10 marks]

Question B5.1 [4 marks]

Answer the following about Java concurrency concepts: [4 marks]

1. Explain what a race condition is. Write a minimal Java code example (3-5 lines) that demonstrates one, then fix it using `synchronized`.
2. What is the difference between `Thread.sleep()` and `Object.wait()`? When would you use each?
3. Explain what the `volatile` keyword does. Give one scenario where you would use it in a Spring Boot application.
4. Describe the `ExecutorService` pattern. Write 4 lines of code showing how to run 3 tasks concurrently and wait for all to finish.

Student answer space:

1. Race Condition

* **Explanation:** A race condition occurs when multiple threads access and modify shared data concurrently, and the final outcome depends on the timing of thread execution.

* **Example & Fix:**

// Problem: Multiple threads calling `increment()` will lose updates

```
class Counter {
    int count = 0;
    synchronized void increment() { count++; } // 'synchronized' fixes the race condition
}
```

2. Thread.sleep() vs Object.wait()

- * **Difference:** Thread.sleep() pauses the thread for a set time but holds onto locks. Object.wait() pauses the thread and releases the lock, allowing other threads to enter the synchronized block.
- * **When to use:** Use sleep() for simple delays. Use wait() for inter-thread communication (e.g., waiting for a specific condition to be met by another thread).

3. The volatile Keyword

- * **Definition:** It ensures "visibility." Changes made to a volatile variable by one thread are immediately visible to all other threads by preventing the CPU from caching the variable locally.
- * **Spring Scenario:** Used for a "Global Shutdown Flag" in a background @Service. If a background task is running, a volatile boolean keepRunning can be toggled by a Controller to stop the service immediately.

4. ExecutorService Pattern

- * **Description:** It is a framework for managing a pool of threads. It decouples task submission from how the task is executed.

Question B5.2 [3 marks]

Java 21 introduced **virtual threads** (Project Loom) and **Records**. (a) Explain what a Record is and convert the following class into a Record. (b) Explain what virtual threads are, how they differ from platform threads, and why they matter for Spring Boot applications. [3 marks]

```
// Convert this to a Java 21 Record:
public final class ProductDTO {
    private final Long id;
    private final String name;
    private final double price;
    private final String category;

    public ProductDTO(Long id, String name, double price, String category) {
        this.id = id; this.name = name;
        this.price = price; this.category = category;
    }
    public Long getId() { return id; }
    public String getName() { return name; }
    public double getPrice() { return price; }
    public String getCategory() { return category; }
}
```

Your solution:

(a) Java Records

- **Definition:** A Record is a special type of class in Java designed to be a transparent carrier for immutable data. It significantly reduces boilerplate code by automatically generating fields, a constructor, equals(), hashCode(), toString(), and accessor methods.
- **Conversion to Record:**

```
public record ProductDTO(Long id, String name, double price, String category) {}
```

(b) Virtual Threads (Project Loom)

- **What they are:** Virtual threads are lightweight threads managed by the Java Virtual Machine (JVM) rather than the operating system.
- **Difference from Platform Threads:** Platform threads are "thin wrappers" around OS threads; they are expensive and limited in number (usually a few thousand). Virtual threads are "mounted" on platform threads only when performing work. When a virtual thread hits a blocking I/O operation (like a database call), it is unmounted, freeing the underlying OS thread for other work.
- **Why they matter for Spring Boot:** They allow Spring Boot applications to handle millions of concurrent connections using the simple "one-thread-per-request" model.

Question B5.3 [3 marks]

Study the following Java 21 code using **sealed classes** and **pattern matching**. Explain what the code does, what would happen if you removed the `sealed` keyword, and complete the missing case in the `describe()` method. [3 marks]

```
public sealed interface Shape permits Circle, Rectangle, Triangle {}

record Circle(double radius) implements Shape {}
record Rectangle(double width, double height) implements Shape {}
record Triangle(double base, double height) implements Shape {}

public static String describe(Shape shape) {
    return switch (shape) {
        case Circle c -> String.format("Circle r=%.2f area=%.2f", c.radius(), Math.PI * c.radius() * c.radius());
        case Rectangle r -> String.format("Rect %sx%s area=%.2f", r.width(), r.height(), r.width() * r.height());
        case Triangle t -> // TODO: complete this case
    };
}
```

Student answer space:


```
2026-04-05T03:23:18.617+03:00 INFO 4952 --- [shopwave-fresh] [main] com.zaxxer.hikari.HikariDataSource : HikariPool-2 - Starting...
2026-04-05T03:23:18.618+03:00 INFO 4952 --- [shopwave-fresh] [main] com.zaxxer.hikari.pool.HikariPool : HikariPool-2 - Added connection conn14; url=jdbc:h2:mem:shopwavedb user=SE
2026-04-05T03:23:18.619+03:00 INFO 4952 --- [shopwave-fresh] [main] com.zaxxer.hikari.HikariDataSource : HikariPool-2 - Start completed.
2026-04-05T03:23:18.619+03:00 WARN 4952 --- [shopwave-fresh] [main] org.hibernate.orm.deprecation : HHH90000025: H2Dialect does not need to be specified explicitly using 'hibernate.dialect' (remove the property setting and it will be selected by default)
2026-04-05T03:23:18.698+03:00 INFO 4952 --- [shopwave-fresh] [main] o.h.e.t.j.p.i.JtaPlatformInitiator : HHH0000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to enable JTA platform integration)
2026-04-05T03:23:18.703+03:00 INFO 4952 --- [shopwave-fresh] [main] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
2026-04-05T03:23:18.818+03:00 WARN 4952 --- [shopwave-fresh] [main] JpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, database queries may be performed during view rendering. Explicitly configure spring.jpa.open-in-view to disable this warning
2026-04-05T03:23:18.921+03:00 INFO 4952 --- [shopwave-fresh] [main] o.s.b.a.h2.H2ConsoleAutoConfiguration : H2 console available at '/h2-console'. Database available at 'jdbc:h2:mem:shopwavedb'
2026-04-05T03:23:18.952+03:00 INFO 4952 --- [shopwave-fresh] [main] com.shopwave.ShopwaveApplicationTests : Started ShopwaveApplicationTests in 0.847 seconds (process running for 8.569)
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.887 s -- in com.shopwave.ShopwaveApplicationTests
[INFO] Results:
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time: 13.137 s
[INFO] Finished at: 2026-04-05T03:23:19+03:00
[INFO] PS C:\Users\pc\Downloads\shopwave-starter\shopwave-fresh-assignment>
```

3.3 Showcasing H2 Database Starts and after I logged in

Run Run Selected Auto complete Clear SQL statement:

jdbc:h2:mem:shopwavedb

- CATEGORIES
- ORDERS
- ORDERS_ITEMS
- ORDER_ITEMS
- PRODUCTS
- INFORMATION_SCHEMA
- Users
- H2 2.2.224 (2023-09-17)

Important Commands

?	Displays this Help Page
↑	Shows the Command History
Ctrl+Enter	Executes the current SQL statement
Shift+Enter	Executes the SQL statement defined by the text selection
Ctrl+Space	Auto complete
⏏	Disconnects from the database

Sample SQL Script

Delete the table if it exists	DROP TABLE IF EXISTS TEST;
Create a new table with ID and NAME columns	CREATE TABLE TEST((ID INT PRIMARY KEY, NAME VARCHAR(255));
Add a new row	INSERT INTO TEST VALUES(1, 'Hello');
Add another row	INSERT INTO TEST VALUES(2, 'World');
Query the table	SELECT * FROM TEST ORDER BY ID;
Change data in a row	UPDATE TEST SET NAME='Hi' WHERE ID=1;
Remove a row	DELETE FROM TEST WHERE ID=2;
Help	HELP ...

Adding Database Drivers

Additional database drivers can be registered by adding the Jar file location of the driver to the environment variables H2DRIVERS or CLASSPATH. Example (Windows): to add the database driver library C:/Programs/hsqldb/lib/hsqldb.jar, set the environment variable H2DRIVERS to C:/Programs/hsqldb/lib/hsqldb.jar.

3.4 Showcasing the POST Product Creation

The screenshot shows an IDE with several tabs open: application.properties, ProductNotFoundException.java, ProductMapper.java, CreateProductRequest.java, and OrderStatus.java. The application.properties file is open, showing the line `spring.application.name=ShopWave`. Below the code editor, the 'Problems' tab is active, displaying an error at line 1, character 25: `+ GET /api/products?page=0&size=10`. The error message states: 'The ampersand (&) character is not allowed. The & operator is reserved for future use; wrap an ampersand in double quotation marks ("&") to pass it as part of a string.' The error details are: `+ CategoryInfo : ParserError: (:) [], ParentContainsErrorRecordException` and `+ FullyQualifiedErrorId : AmpersandNotAllowed`.

Below the error, a PowerShell terminal window shows a successful REST client invocation: `Invoke-RestMethod -Method Post -Uri "http://localhost:8080/api/products" -ContentType "application/json" -Body '{"name":"Dell Inspiron","description":"Student Laptop","price":850,"stock":12,"categoryId":1}'`. The response is displayed as a JSON object: `{ id: 2, name: Dell Inspiron, description: Student Laptop, price: 850, stock: 12, categoryId: 1 }`.

The terminal window is titled `PS C:\Users\pc\Downloads\shopwave-starter\shopwave-fresh-assignment>`.

3.5 Product Retrieval - Showcasing Pagination

```
application.properties M X ProductNotFoundException.java U ProductMapper.java U CreateProductRequest.java U OrderStatus.java U
src > main > resources > application.properties
1 | spring.application.name=ShopWave

Problems 175 Output Debug Console Terminal Ports
+ v ... | | | x

git
powershell
java
powershell

content      : {@{id=1; name=Dell Inspiron; description=Student laptop; price=850.00; stock=10; categoryId=1}, @{id=2; name=Dell Inspiron;
description=Student Laptop; price=850.00; stock=12; categoryId=1}}
pageable     : {@(pageNumber=0; pageSize=10; sort=; offset=0; unpaged=False; paged=True}
last         : True
totalElements : 2
totalPages   : 1
size         : 10
number       : 0
sort         : {@(empty=True; sorted=False; unsorted=True}
first        : True
numberOfElements : 2
empty        : False

PS C:\Users\pc\Downloads\shopwave-starter\shopwave-fresh-assignment> Invoke-RestMethod -Method Get -Uri "http://localhost:8080/api/products?page
=0&size=10"

content      : {@{id=1; name=Dell Inspiron; description=Student laptop; price=850.00; stock=10; categoryId=1}, @{id=2; name=Dell Inspiron;
description=Student Laptop; price=850.00; stock=12; categoryId=1}}
pageable     : {@(pageNumber=0; pageSize=10; sort=; offset=0; unpaged=False; paged=True}
last         : True
totalElements : 2
totalPages   : 1
size         : 10
number       : 0
sort         : {@(empty=True; sorted=False; unsorted=True}
first        : True
numberOfElements : 2
empty        : False
```

3.6 Showcasing Search Functionality

```
application.properties M X ProductNotFoundException.java U ProductMapper.java U CreateProductRequest.java U OrderStatus.java U
src > main > resources > application.properties
1 | spring.application.name=ShopWave

Problems 175 Output Debug Console Terminal Ports
+ v ... | | | x

git
powershell
java
powershell

id           : 1
name          : Dell Inspiron
description   : Student laptop
price         : 850.00
stock         : 10
categoryid    : 1

id           : 2
name          : Dell Inspiron
description   : Student Laptop
price         : 850.00
stock         : 12
categoryid    : 1

PS C:\Users\pc\Downloads\shopwave-starter\shopwave-fresh-assignment> |
```

3.7 Showcasing 404 Error Handling

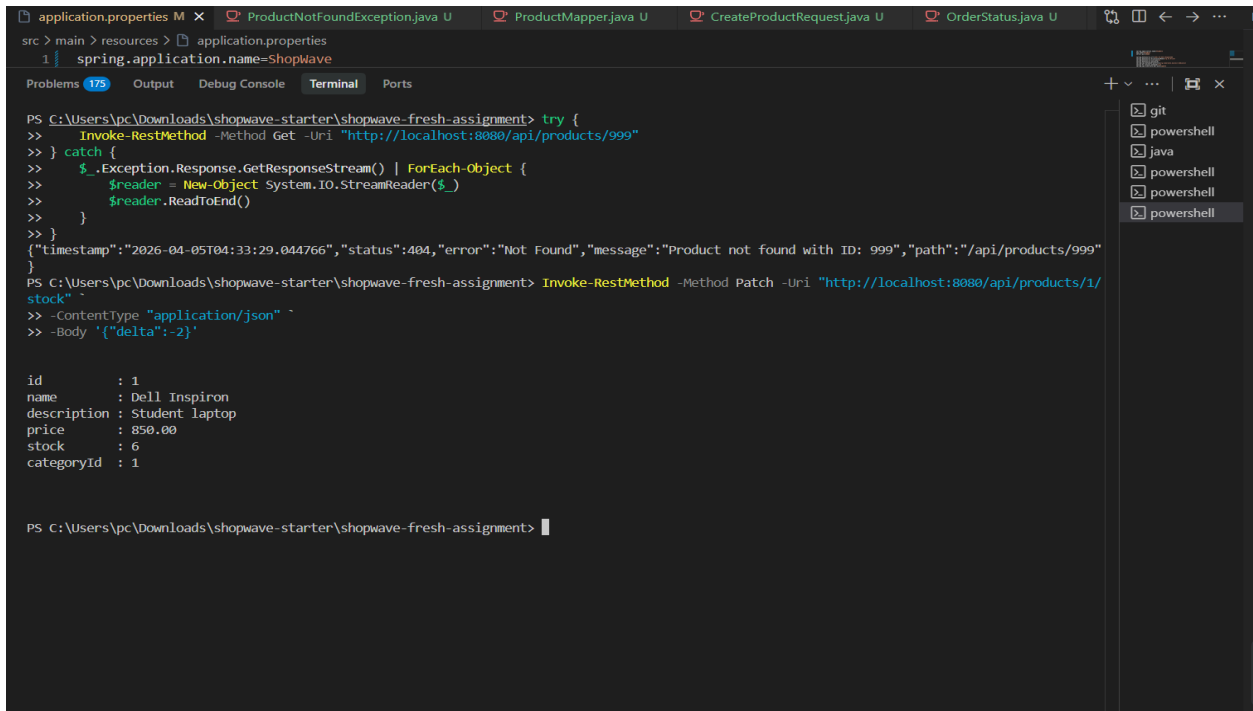
```
application.properties M X ProductNotFoundException.java U ProductMapper.java U CreateProductRequest.java U OrderStatus.java U
src > main > resources > application.properties
1 | spring.application.name=ShopWave

Problems 175 Output Debug Console Terminal Ports
+ v ... | | | x

git
powershell
java
powershell
powershell
powershell

PS C:\Users\pc\Downloads\shopwave-starter\shopwave-fresh-assignment> try {
>>     Invoke-RestMethod -Method Get -Uri "http://localhost:8080/api/products/999"
>> } catch {
>>     $_.Exception.Response.GetResponseStream() | ForEach-Object {
>>         $reader = New-Object System.IO.StreamReader($_)
>>         $reader.ReadToEnd()
>>     }
>> }
{"timestamp":"2026-04-05T04:33:29.044766","status":404,"error":"Not Found","message":"Product not found with ID: 999","path":"/api/products/999"}
PS C:\Users\pc\Downloads\shopwave-starter\shopwave-fresh-assignment> |
```

3.8 Showcasing Stock Validation



```
src > main > resources > application.properties
1 | spring.application.name=ShopWave

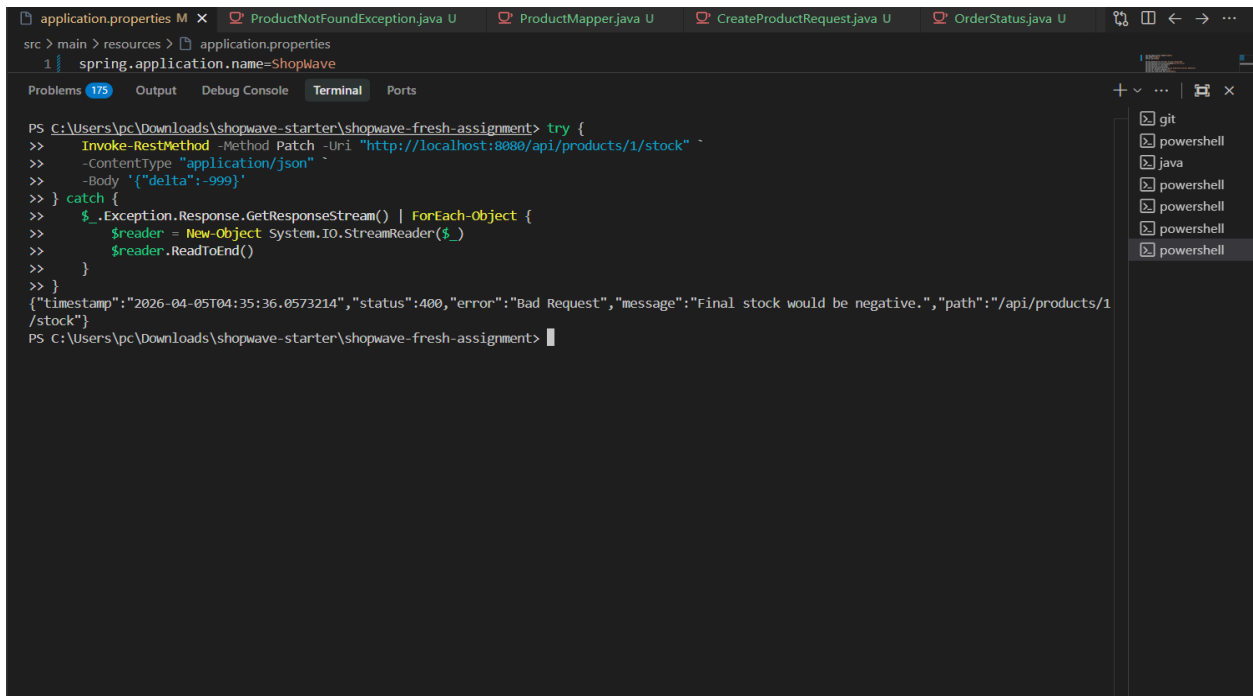
Problems 175 Output Debug Console Terminal Ports

PS C:\Users\pc\Downloads\shopwave-starter\shopwave-fresh-assignment> try {
>>   Invoke-RestMethod -Method Get -Uri "http://localhost:8080/api/products/999"
>> } catch {
>>   $_.Exception.Response.GetResponseStream() | ForEach-Object {
>>     $reader = New-Object System.IO.StreamReader($_)
>>     $reader.ReadToEnd()
>>   }
>> }
{"timestamp":"2026-04-05T04:33:29.044766","status":404,"error":"Not Found","message":"Product not found with ID: 999","path":"/api/products/999"}
PS C:\Users\pc\Downloads\shopwave-starter\shopwave-fresh-assignment> Invoke-RestMethod -Method Patch -Uri "http://localhost:8080/api/products/1/stock" -
>> -ContentType "application/json" -
>> -Body '{"delta":-2}'

id      : 1
name    : Dell Inspiron
description : Student laptop
price   : 850.00
stock   : 6
categoryId : 1

PS C:\Users\pc\Downloads\shopwave-starter\shopwave-fresh-assignment>
```

3.9 Showcasing Invalid stock update



```
src > main > resources > application.properties
1 | spring.application.name=ShopWave

Problems 175 Output Debug Console Terminal Ports

PS C:\Users\pc\Downloads\shopwave-starter\shopwave-fresh-assignment> try {
>>   Invoke-RestMethod -Method Patch -Uri "http://localhost:8080/api/products/1/stock" -
>>   -ContentType "application/json" -
>>   -Body '{"delta":-999}'
>> } catch {
>>   $_.Exception.Response.GetResponseStream() | ForEach-Object {
>>     $reader = New-Object System.IO.StreamReader($_)
>>     $reader.ReadToEnd()
>>   }
>> }
{"timestamp":"2026-04-05T04:35:36.0573214","status":400,"error":"Bad Request","message":"Final stock would be negative.","path":"/api/products/1/stock"}
PS C:\Users\pc\Downloads\shopwave-starter\shopwave-fresh-assignment>
```